



40-MAVZU · V BOB. DRF VA DEPLOY

DRF yordamida REST API yaratish. CRUD API ishlab chiqish

Generic view'lar (ListCreateAPIView, RetrieveUpdateDestroyAPIView), ViewSet va Router, ruxsatlar (faqat muallif), sahifalash, qidiruv va saralash, token autentifikatsiya, CORS; Front-end (JavaScript fetch) va Back-end integratsiyasi; mahsulotlar API'si.

8-sinf · Web Full-stack dasturlash

Bugun nimani o'rganamiz?

- To'liq **CRUD API** — bir necha qatorda
- **ViewSet** va **Router** — URL'lar avtomatik
- **Ruxsatlar**: faqat muallif o'zgartiradi
- **Sahifalash, qidiruv, saralash**, token
- **Front-end + Back-end integratsiyasi** (29-loyiha)
- Mahsulotlar boshqaruv API'si (27–28-loyihalar)

Abstraksiya zinapoyasi

Daraja	Vosita	Kod	Qachon
1	<code>@api_view</code> / <code>APIView</code>	Ko'p, hammasi qo'lda	Nostandart mantiq
2	Generic view'lar	Kam	Standart CRUD, URL'lar sizda
3	<code>ModelViewSet</code> + Router	Eng kam	To'liq standart CRUD



Ovqat tayyorlash: 1 — noldan pishirish; 2 — yarim tayyor mahsulotlardan; 3 — tayyor taomni isitish. Maxsus taom uchun — noldan, oddiy tushlik uchun — tayyoridan.

2-daraja: Generic view'lar

blog/api_views.py

```
from rest_framework import generics

class MaqolaListCreate(generics.ListCreateAPIView):           # GET ro'yxat + POST
    queryset = Maqola.objects.filter(chop_etilgan=True).select_related("kategoriya", "muallif")
    serializer_class = MaqolaSerializer

    def perform_create(self, serializer):
        serializer.save(muallif=self.request.user)             # muallifni qo'shamiz

class MaqolaDetail(generics.RetrieveUpdateDestroyAPIView):    # GET + PUT + PATCH + DELETE
    queryset = Maqola.objects.all()
    serializer_class = MaqolaSerializer
```

blog/urls.py

```
path("api/maqolalar/", api_views.MaqolaListCreate.as_view()),
path("api/maqolalar/<int:pk>/", api_views.MaqolaDetail.as_view()), # generic'lar "pk" kutadi
```

 ~15 qatorda to'liq CRUD! Lekin hozir **istalgan** foydalanuvchi begona maqolani o'zgartira oladi...

Ruxsatlar (permissions)

blog/permissions.py

```
from rest_framework import permissions

class MuallifYokiFaqatOqish(permissions.BasePermission):
    """O'qish — hammaga; o'zgartirish — faqat muallifga."""
    def has_object_permission(self, request, view, obj):
        if request.method in permissions.SAFE_METHODS: # GET, HEAD, OPTIONS
            return True
        return obj.muallif == request.user
```

Python

```
permission_classes = [permissions.IsAuthenticatedOrReadOnly, MuallifYokiFaqatOqish]
```

Tayyor ruxsat	Ma'nosi
<code>AllowAny</code>	Hammaga ochiq
<code>IsAuthenticated</code>	Faqat kirganlar
<code>IsAuthenticatedOrReadOnly</code>	O'qish — hammaga, yozish — kirganlarga
<code>IsAdminUser</code>	Faqat admin

- Begona PUT/DELETE → **403 Forbidden** (38-mavzu qoidasi, API'da)

3-daraja: ViewSet

Python

```
from rest_framework import viewsets, permissions, filters

class MaqolaViewSet(viewsets.ModelViewSet):    # list, create, retrieve, update, destroy
    serializer_class = MaqolaSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly, MuallifYokiFaqatOqish]
    filter_backends = [filters.SearchFilter, filters.OrderingFilter]
    search_fields = ["sarlavha", "matn"]        # ?search=futbol
    ordering_fields = ["yaratilgan", "korishlar"]    # ?ordering=-korishlar

    def get_queryset(self):
        qs = Maqola.objects.filter(chop_etilgan=True).select_related("kategoriya", "muallif")
        kategoriya = self.request.query_params.get("kategoriya")    # ?kategoriya=sport
        if kategoriya:
            qs = qs.filter(kategoriya__slug=kategoriya)
        return qs

    def perform_create(self, serializer):
        serializer.save(muallif=self.request.user)
```

Router — URL'lar avtomatik

config/urls.py

```
from rest_framework.routers import DefaultRouter
from blog.api_views import MaqolaViewSet, KategoriyaViewSet

router = DefaultRouter()
router.register("maqolalar", MaqolaViewSet, basename="maqola")
router.register("kategoriyalar", KategoriyaViewSet)

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include(router.urls)),
    path("api-auth/", include("rest_framework.urls")),
    path("", include("blog.urls")),
]
```

URL	Metodlar	ViewSet metodi
/api/maqolalar/	GET, POST	list, create
/api/maqolalar/5/	GET, PUT, PATCH, DELETE	retrieve, update, partial_update, destroy



Oldingi qo'lda yozilgan `api/maqolalar/...` yo'llarini o'chiring — ikki joyda bo'lmasin

Sahifalash

settings.py

```
REST_FRAMEWORK = {  
    "DEFAULT_PERMISSION_CLASSES": ["rest_framework.permissions.IsAuthenticatedOrReadOnly"],  
    "DEFAULT_PAGINATION_CLASS": "rest_framework.pagination.PageNumberPagination",  
    "PAGE_SIZE": 10,  
}
```

JSON

```
{  
    "count": 23,  
    "next": "http://127.0.0.1:8000/api/maqolalar/?page=3&search=maktab",  
    "previous": "http://127.0.0.1:8000/api/maqolalar/?page=1&search=maktab",  
    "results": [  
        { "id": 12, "sarlavha": "Maktab kutubxonasi yangilandi", "korishlar": 340 }  
    ]  
}
```

Token autentifikatsiya

- Brauzer — sessiya (cookie); mobil ilova va dasturlar — har so'rovda **token**

Python

```
INSTALLED_APPS += ["rest_framework.authtoken"]
REST_FRAMEWORK["DEFAULT_AUTHENTICATION_CLASSES"] = [
    "rest_framework.authentication.SessionAuthentication",
    "rest_framework.authentication.TokenAuthentication",
]
# urls.py: path("api/token/", obtain_auth_token)
```

Terminal

```
curl -X POST http://127.0.0.1:8000/api/token/ -d "username=ali&password=parol12345"
# {"token": "9944b09199c62bcf9418ad846dd0e4bbdfc6ee4b"}

curl -X POST http://127.0.0.1:8000/api/maqolalar/ \
-H "Authorization: Token 9944b09199c62bcf..." \
-H "Content-Type: application/json" \
-d '{"sarlavha": "Token bilan yozildi", "matn": "Salom!", "kategoriya": 1}'
```



Token — parol kabi **maxfiy**! GitHub'ga yuklamang; faqat HTTPS orqali

BO'LIM

Front-end + Back-end

Kursning eng muhim lahzasi: II bobdagi JavaScript va Django API birlashadi

CORS — boshqa saytdan so'rovlarga ruxsat

- Brauzer bir saytdagi JS'ga **boshqa domendagi** API'ga so'rov yuborishni taqiqlaydi
- Ruxsatni **server** beradi:

Terminal

```
pip install django-cors-headers
```

settings.py

```
INSTALLED_APPS += ["corsheaders"]
MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",      # ← eng boshiga
    "django.middleware.security.SecurityMiddleware",
    # ...
]
CORS_ALLOWED_ORIGINS = [
    "http://127.0.0.1:5500",                       # VS Code Live Server
    "https://maktab-blog.netlify.app",             # internetdagi front-end
]
```

Front-end: JavaScript mijoz (29-loyiha)

JavaScript

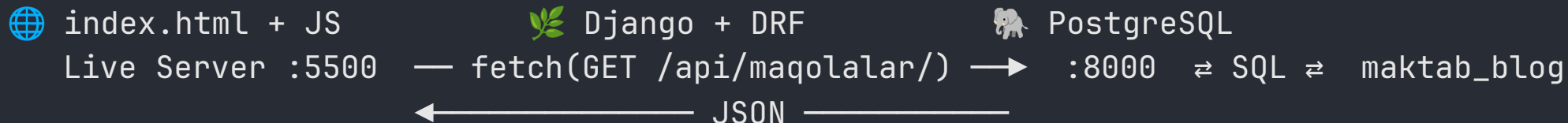
```
const API = "http://127.0.0.1:8000/api/maqolalar/";

async function yukla(url) {
  holat.textContent = "Yuklanmoqda...";
  try {
    const javob = await fetch(url);
    if (!javob.ok) throw new Error(`Server xatosi: ${javob.status}`);
    const data = await javob.json(); // { count, next, previous, results }
    royxat.innerHTML = "";
    data.results.forEach(m => {
      const karta = document.createElement("article");
      karta.className = "karta";
      const h3 = document.createElement("h3");
      h3.textContent = m.sarlavha; // textContent - XSS'dan himoya
      karta.append(h3);
      royxat.append(karta);
    });
    holat.textContent = `Jami: ${data.count} ta maqola`;
  } catch (err) {
    holat.textContent = `❌ ${err.message}. Django server ishlayaptimi?`;
  }
}

yukla(API);
```

Full-stack zanjiri

Matn



Siz full-stack ilovaning barcha qismlarini o'zingiz yozdingiz! 🎉

- Front-end: HTML, CSS, JavaScript (I–II bob)
- Back-end: Python, Django, DRF (IV–V bob)
- Ma'lumotlar bazasi: PostgreSQL, SQL (III bob)

Amaliyot: mahsulotlar API (27–28-loyihalar)

shop/models.py

```
class Mahsulot(models.Model):
    nomi = models.CharField(max_length=200)
    tavsif = models.TextField(blank=True)
    narx = models.DecimalField(max_digits=12, decimal_places=2)
    soni = models.PositiveIntegerField(default=0)
    faol = models.BooleanField(default=True)
    yaratilgan = models.DateTimeField(auto_now_add=True)
```

- 1 `MahsulotSerializer` (`validate_narx` — musbat), `MahsulotViewSet`, router
- 2 Ruxsat: o'qish — hammaga, o'zgartirish — faqat admin
- 3 `?narx_min=10000&narx_max=50000` filtri
- 4 GET, POST, PATCH, DELETE ni sinang
- 5 Front-end: mahsulotlar katalogi — ma'lumot API'dan

Amaliyot: ruxsat uchun maslahat

- Ro'yxat va bitta mahsulotni ko'rish — hammaga, qolgan amallar — faqat admin:

Python

```
def get_permissions(self):  
    if self.action in ("list", "retrieve"):  
        return [permissions.AllowAny()]  
    return [permissions.IsAdminUser()]
```

Ko'p uchraydigan xatolar

- ❌ `blocked by CORS policy` — corsheaders yo'q, middleware boshida emas yoki manzil (port ham!) ro'yxatda yo'q
- ❌ `get_queryset` bilan ViewSet'da router `basename` unutilgan
- ❌ Generic view URL'ida `<int:id>` — `pk` kerak
- ❌ `fetch` natijasini `.json()` siz yoki `await` siz ishlatish

Savol

Bitta klassda list, create, retrieve, update, destroy beruvchi DRF klassi?

A APIView

B ListAPIView

C ModelViewSet

D Serializer

Javobni tanlang — to'g'ri variant yashil bo'ladi

Savol

ViewSet uchun URL'larni avtomatik yaratuvchi vosita?

A Router (DefaultRouter)

B path()

C Paginator

D Middleware

Javobni tanlang — to'g'ri variant yashil bo'ladi

Savol

SAFE_METHODS qaysi metodlarni o'z ichiga oladi?

A POST, PUT

B DELETE

C Hammasi

D GET, HEAD, OPTIONS

Javobni tanlang — to'g'ri variant yashil bo'ladi

Savol

Netlify'dagi JS sayt Django API'ga so'rovda "CORS" xatosi chiqdi. Nima qilish kerak?

- A** JavaScript kodini o'chirish
- B** `django-cors-headers` ni sozlab, front-end manzilini ruxsat etish
- C** Brauzerni almashtirish
- D** API'ni HTML'ga aylantirish

Javobni tanlang — to'g'ri variant yashil bo'ladi



Uyga vazifa

- 1 JS front-end'ga "**Maqola qo'shish**" formasi — token bilan POST
- 2 Kutubxona API'sini **ViewSet + Router**'ga o'tkazing, qidiruv va sahifalash

JavaScript

```
fetch(API, {  
  method: "POST",  
  headers: { "Authorization": "Token ...", "Content-Type": "application/json" },  
  body: JSON.stringify({ sarlavha, matn, kategoriya: 1 }),  
});
```

Yodda tuting

- Generic: `ListCreateAPIView`, `RetrieveUpdateDestroyAPIView`; `perform_create`
- `ModelViewSet` + `DefaultRouter` — to'liq CRUD API
- `permission_classes`, o'z `BasePermission` (`has_object_permission`)
- `SearchFilter`, `OrderingFilter`, `PageNumberPagination`; token
- Front-end `fetch` ⇔ DRF API; CORS — `django-cors-headers`



Savollar bormi?

Keyingi mavzu: **41. Django loyihasini serverga joylash (Deploy)**

Batafsil matn, misollar va testlar: **40-drf-crud-api.html** darsida